

Stredná priemyselná škola informačných technológií
Nábřežná 1325, 024 01 Kysucké Nové Mesto

WEBOVÁ APLIKÁCIA

Stredoškolská odborná činnosť

Č. odboru: 11 – informatika

Miesto: Kysucké Nové Mesto

Rok: 2025

Riešitelia: Ronnie Rybárik

Ročník štúdia: štvrtý

Stredná priemyselná škola informačných technológií
Nábřežná 1325, 024 01 Kysucké Nové Mesto

WEBOVÁ APLIKÁCIA

Stredoškolská odborná činnosť

Č. odboru: 11 – informatika

Miesto: Kysucké Nové Mesto

Rok: 2025

Riešitelia: Ronnie Rybárik

Ročník štúdia: štvrtý

Školiteľ: Mgr. Zuzana Polláková

Čestné vyhlásenie

Vyhlasujem, že prácu stredoškolskej odbornej činnosti na tému „Webová aplikácia Pre vodičské skúšky“ som vypracoval samostatne, s použitím uvedených literárnych zdrojov. Prácu som neprihlásil a ani neprezentoval v žiadnej inej súťaži, ktorá je pod gestorstvom MŠVVaM SR. Som si vedomý dôsledkov, ak uvedené údaje nie sú pravdivé.

V Kysuckom Novom Meste, dňa

.....

podpis

ABSTRAKT

Naša práca sa zameriava na vývoj webovej aplikácie, ktorá efektívne podporuje prípravu na vodičské skúšky na Slovensku. Používateľom ponúka simulované inteligentné testy s bodovaním pre všetky kategórie vodičských oprávnení (A, B, BE, C, CE, D, DE, T), možnosť sledovať svoj pokrok a analyzovať nesprávne odpovede. Aplikácia obsahuje aj simulátor križovatiek, rozhranie na učenie dopravných značiek a profily používateľov s prehľadom ich výsledkov a históriou testov. Táto aplikácia umožňuje správcovi mať prehľad nad úspešnosťami testov, meniť a pridávať otázky a kontrolovať pripomienky užívateľov.

Kľúčové slová: Webová aplikácia ,Vodičské skúšky, Inteligentné testy, Príprava

Rozsah: 19 s. vrátane príloh, z toho 18 s. textovej časti

ABSTRACT

Our work focuses on developing a web application that effectively supports preparation for driving exams in Slovakia. It offers users simulated intelligent tests with scoring for all categories of driving licenses (A, B, BE, C, CE, D, DE, T), the ability to track their progress, and analyze incorrect answers. The application also includes an intersection simulator, an interface for learning traffic signs, and user profiles with an overview of their results and test history. This application allows the administrator to monitor test success rates, modify and add questions, and manage user feedback.

Keywords: Web application, Driving exams, Intelligent tests, Preparation

Size: 19p. including appendix, 18 p. of main part

OBSAH

0	Úvod	7
1	Problematika a prehľad literatúry	8
1.1	Architektúra aplikácie.....	8
1.2	Frontend - React a TypeScript.....	8
1.3	Backend - Node.js a Express	9
1.4	Databázové riešenie - Supabase, Prisma a PostgreSQL	10
1.5	Ďalšie použité nástroje.....	11
2	Ciele práce	13
3	Materiál a metodika	14
3.1	Vytvorenie projektu a základné nastavenia aplikácie.....	14
3.2	Integrácia frontendu a backendu.....	16
3.3	Vytváranie prvkov a funkcií aplikácie.....	16
3.4	Hosting a Deployment	19
4	Výsledky práce a diskusia	21
5	Závery práce	22
6	Zhrnutie	23
	Zoznam použitej literatúry.....	24

ZOZNAM TABULIEK, GRAFOV A ILUSTRÁCIÍ

ZOZNAM ILUSTRÁCIÍ

Obrázok 1 Logo React + Typescript	9
Obrázok 2 Logo Node.js + Express.....	9
Obrázok 3 Supabase grafické rozhranie tabuliek	10
Obrázok 4 Logo git + Github	12
Obrázok 5 Ukážka tailwind.config.js	14
Obrázok 6 Ukážka schema.prisma	15
Obrázok 7 Ukážka express routov	16
Obrázok 8 Vyhodnotenie testu s možnosťou prezerania otázok	17
Obrázok 9 Ukážka admin dashboardu.....	19
Obrázok 10 DNS konfigurácia na platforme Namecheap	19

0 ÚVOD

Dopravná bezpečnosť a znalosť pravidiel cestnej premávky sú kľúčové pre bezpečné a zodpovedné správanie vodičov na cestách. Vodičské skúšky predstavujú dôležitý krok v tomto procese, no mnohí uchádzači čelia problémom pri príprave. Často sa spoliehajú na statické učebnice alebo neaktuálne testy, ktoré neodrážajú reálne situácie na cestách. Tento problém nás viedol k vytvoreniu webovej aplikácie, ktorá moderným a interaktívnym spôsobom pomáha používateľom efektívne sa pripraviť na vodičské skúšky na Slovensku.

Naším cieľom bolo navrhnuť platformu, ktorá kombinuje simulované testy, interaktívny simulátor križovatiek a precvičovanie dopravných značiek. Chceli sme poskytnúť používateľom inteligentný systém, ktorý dokáže analyzovať ich chyby a prispôbiť testy tak, aby sa zamerali na problematické oblasti. Okrem toho sme chceli zabezpečiť jednoduché a prehľadné užívateľské rozhranie, ktoré umožní efektívne učenie bez zbytočného rozptyľovania.

Táto práca sa zameriava na proces vývoja aplikácie od návrhu po implementáciu. V teoretickej časti sa venujeme problematike vývoja webových aplikácií, popisujeme architektúru systému a technológie, ktoré sme použili. V praktickej časti detailne rozoberáme implementáciu kľúčových funkcií, ako je generovanie testov, simulátor križovatiek či administrátorský panel na správu otázok a štatistík.

Výsledkom tejto práce je funkčná webová aplikácia, ktorá poskytuje komplexný nástroj na prípravu na vodičské skúšky. Veríme, že náš prístup modernizuje spôsob učenia a prispeje k lepšej pripravenosti budúcich vodičov.

1 PROBLEMATIKA A PREHLAD LITERATÚRY

Webová aplikácia, ktorú sme vytvorili, je výsledkom integrácie moderných nástrojov a technológií, ktoré spoločne vytvárajú efektívne, robustné a škálovateľné riešenie. Na vývoj sme využili React pre frontend, TypeScript na zabezpečenie typovej bezpečnosti, Node.js a Express na spracovanie backendu, Supabase ako databázovú službu a Prisma ORM na správu databázy PostgreSQL. Každý z týchto nástrojov sme starostlivo vybrali pre ich konkrétne výhody a prínosy k celkovému riešeniu. V tejto kapitole popíšeme architektúru našej aplikácie a jednotlivé technológie, ktoré sme implementovali.

1.1 Architektúra aplikácie

Aplikácia je navrhnutá na princípe oddelenej architektúry, známej tiež ako viacvrstvová architektúra. Tento model zabezpečuje, že frontend, backend a databáza fungujú ako samostatné moduly, pričom medzi sebou komunikujú prostredníctvom REST API. Frontend sprístupňuje používateľom vizuálne a interaktívne rozhranie, backend spracúva logiku aplikácie a zodpovedá za všetky požiadavky používateľov, zatiaľ čo databáza uchováva všetky dôležité dáta v štruktúrovanej forme.

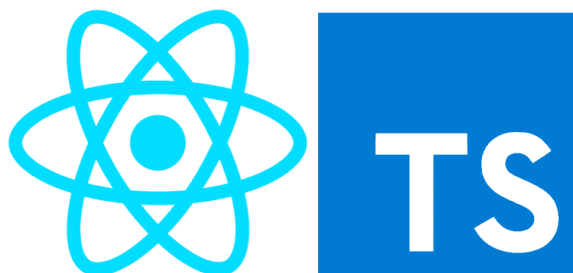
Architektúra bola navrhnutá s dôrazom na modularitu (každá vrstva je nezávislá, čo zjednodušuje údržbu a škálovanie aplikácie), škálovateľnosť (možnosť rozšíriť jednotlivé časti aplikácie bez ovplyvnenia ostatných) a bezpečnosť (jasné oddelenie frontendu a backendu znižuje riziko neoprávneného prístupu).

1.2 Frontend - React a TypeScript

Frontend aplikácie bol vyvinutý pomocou Reactu, čo nám umožnilo využiť výhody komponentovej architektúry. Každá časť používateľského rozhrania, ako napríklad tlačidlá, formuláre alebo tabuľky, bola implementovaná ako samostatný znovupoužiteľný komponent aj pomocou knižnice Shadcn. Tento prístup zjednodušuje nielen samotný vývoj, ale aj neskoršiu údržbu a rozširovanie aplikácie.

Použitie TypeScriptu viedlo k výraznému zlepšeniu typovej bezpečnosti kódu. TypeScript nám umožnil definovať presné typy dát, ktoré jednotlivé komponenty očakávajú, čo minimalizovalo množstvo runtime chýb. Táto typová bezpečnosť zlepšila aj komunikáciu s API, keďže sme vedeli presne definovať štruktúru dát, ktoré boli

odosielané a prijímané. Výhody použitia Reactu a TypeScriptu v našej aplikácii sú komponentová architektúra (všetky prvky rozhrania sú modulárne a jednoducho znovupoužiteľné), reaktivita (používateľské rozhranie sa dynamicky prispôsobuje zmenám dát) a typová bezpečnosť (Znižuje riziko chýb pri práci s komplexnými dátami a API).



Obrázok 1 Logo React + Typescript

1.3 Backend - Node.js a Express

Backend aplikácie sme postavili na platforme Node.js, ktorá je známa svojou vysokou výkonnosťou a škálovateľnosťou. Ako framework sme použili Express, ktorý nám umožnil rýchlo a efektívne implementovať API a middleware. Backendová časť aplikácie spracúva všetky požiadavky od frontendu a poskytuje údaje z databázy prostredníctvom REST API. Implementovali sme funkcie na autentifikáciu a autorizáciu používateľov, validáciu vstupných dát a prácu s databázou. Middleware, ktorý sme použili, umožnilo efektívne spracovať chyby a zabezpečiť bezpečnú komunikáciu.

Hlavné výhody Node.js a Express sú asynchrónne spracovanie (umožňuje efektívne zvládať viacero požiadaviek súčasne), flexibilita (jednoduché prispôsobenie a rozšírenie backendovej logiky), škálovateľnosť (Node.js je ideálny pre aplikácie, ktoré musia spracovať veľké množstvo požiadaviek).



Obrázok 2 Logo Node.js + Express

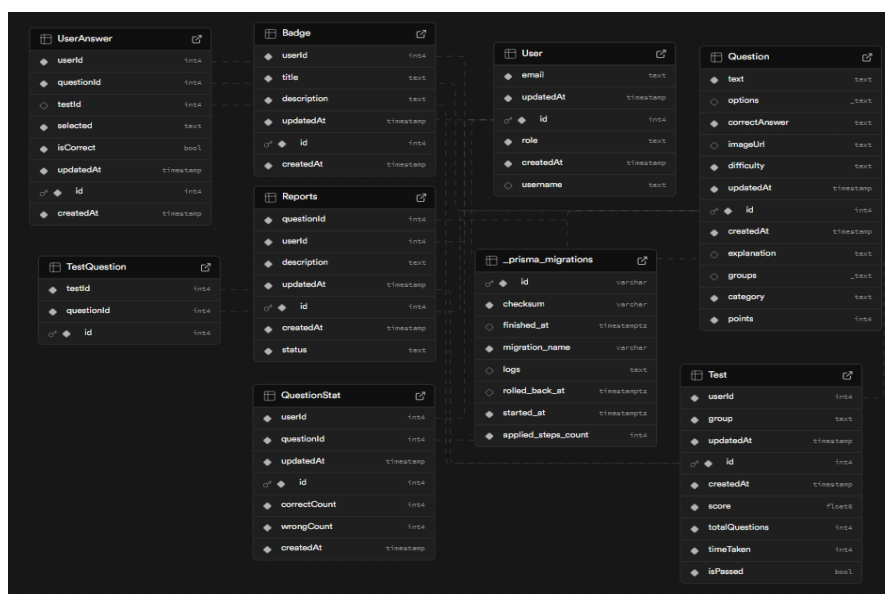
1.4 Databázové riešenie - Supabase, Prisma a PostgreSQL

Správa dát bola jedným z hlavných pilierov našej aplikácie. Preto sme sa rozhodli použiť moderné databázové riešenia, ktoré zabezpečujú spoľahlivosť, výkon a škálovateľnosť.

Ako hlavný databázový systém sme použili PostgreSQL, ktorý je jednou z najspoľahlivejších relačných databáz. PostgreSQL umožňuje efektívnu manipuláciu s relačnými dátami a podporuje pokročilé funkcie, ako sú komplexné dotazy, transakcie a indexy. Táto databáza nám poskytla stabilné riešenie pre ukladanie a správu všetkých dát, vrátane používateľských účtov, výsledkov testov a ďalších dôležitých údajov.

Pre jednoduchšiu prácu s databázou sme použili Prisma ORM. Tento nástroj umožňuje definovanie dátových modelov - všetky tabuľky a ich vzťahy sú presne definované pomocou schémy, automatizáciu dotazov - Prisma generuje SQL dotazy, čo výrazne šetrí čas a znižuje riziko chýb a typovú bezpečnosť - typy definované v schéme sú automaticky dostupné v TypeScripte.

Na správu a hostovanie databázy PostgreSQL sme využili službu Supabase. Táto platforma nám poskytla robustné riešenie pre správu dát, API prístup a bezpečnostné funkcie. Vďaka Supabase sme dokázali rýchlo nasadiť databázové riešenie bez potreby zložitej konfigurácie.



Obrázok 3 Supabase grafické rozhranie tabuliek

1.5 Ďalšie použité nástroje

Pri vývoji našej aplikácie sme využili niekoľko nástrojov, ktoré nám výrazne uľahčili prácu a zabezpečili vysokú kvalitu výsledného riešenia. Okrem hlavných technológií sme implementovali nástroje na správu kódu, dizajn, testovanie a vizualizáciu. Detailnejšie popíšeme použitie Git, GitHub, ShadCN, Tailwind CSS a Three.js.

Git sme použili ako verzovací systém, ktorý nám poskytoval bezpečný spôsob na správu zmien v kóde. Tento nástroj nám umožnil nielen zaznamenávať každý krok vo vývoji, ale tiež zabezpečiť možnosť návratu k predchádzajúcim verziám kódu v prípade problémov. Výhodou Gitu bolo aj to, že nám umožnil efektívne spolupracovať na rôznych častiach projektu bez rizika konfliktov. Git bol neoddeliteľnou súčasťou každodenného vývoja, pretože sme mohli detailne sledovať históriu zmien a udržiavať štruktúru kódu čistú a prehľadnú.

Spolu s Gitom sme využívali GitHub, ktorý slúžil ako hostingová platforma pre náš kód. GitHub poskytoval vzdialené repozitáre, ktoré nám umožnili synchronizáciu zmien medzi viacerými zariadeniami a členmi tímu. Prostredníctvom funkcií, ako sú pull requesty, sme mohli kontrolovať a schvaľovať zmeny pred ich implementáciou do hlavnej vetvy. GitHub nám navyše poskytol priestor na dokumentáciu projektu pomocou wiki a správu úloh pomocou systému issues, čo zjednodušilo koordináciu a plánovanie.

Na tvorbu používateľského rozhrania sme využili ShadCN, modernú knižnicu komponentov postavenú na Tailwind CSS. ShadCN nám ponúkol širokú škálu pripravených komponentov, ako sú tlačidlá, formuláre, tabuľky či modálne okná, ktoré sme mohli ľahko prispôbiť našim potrebám. Táto knižnica nám pomohla zabezpečiť konzistenciu v dizajne a zároveň výrazne zrýchlila proces vývoja, keďže sme sa nemuseli zaoberať implementáciou základných prvkov od nuly.

V rámci štylovania celej aplikácie sme použili Tailwind CSS, ktorý je utility-first frameworkom pre tvorbu moderného webového dizajnu. Tento nástroj nám umožnil písať štýly priamo v HTML, čo viedlo k rýchlej a intuitívnej práci. Tailwind CSS nám zároveň poskytol flexibilitu pri prispôbovaní farebných schém, typografie a responzívneho rozloženia. Vďaka nemu sme mohli vytvárať jednotný vzhľad aplikácie a zároveň zabezpečiť, že bude optimalizovaná pre rôzne zariadenia a veľkosti obrazoviek.

Ďalším významným nástrojom, ktorý sme využili, bola knižnica Three.js. Táto knižnica nám umožnila integrovať do aplikácie 3D modely a vizualizácie, čím sme dosiahli modernejší a pútavejší vzhľad. Pomocou Three.js sme mohli zobraziť interaktívne trojrozmerné objekty, ktoré sme doplnili animáciami a realistickými efektmi. Táto funkcionality výrazne obohatila používateľskú skúsenosť a poskytla aplikácii nový rozmer.

Všetky tieto nástroje spoločne vytvorili robustný ekosystém, ktorý nám umožnil efektívne a precízne vyvíjať našu aplikáciu. Integrácia týchto nástrojov do nášho vývojového procesu bola rozhodujúcim krokom pre dosiahnutie kvalitného výsledku, ktorý spĺňa všetky požiadavky moderných webových aplikácií.



Obrázok 4 Logo git + Github

2 CIELE PRÁCE

Hlavné ciele:

- Vytvorenie komplexnej webovej aplikácie na efektívnu prípravu používateľov na vodičské skúšky na Slovensku
- Navrhnutie a implementovanie inteligentného testovacieho systému, ktorý dokáže analyzovať chyby používateľov a prispôsobovať testy na základe ich výkonu
- Zabezpečenie prehľadného, intuitívneho a jednoduchého užívateľského rozhrania, ktoré umožní jednoduché a efektívne učenie

Vedľajšie ciele:

- Vytvorenie administrátorského rozhrania, ktoré umožní správu testov, sledovanie štatistík a spracovanie spätnej väzby od používateľov
- Poskytnutie užívateľom možnosť sledovania ich pokroku, vrátane histórie testov, úspešnosti a analýzy chýb, aby sa mohli lepšie pripraviť na skúšky

3 MATERIÁL A METODIKA

V tejto kapitole sa venujeme podrobnému opisu postupov, ktoré sme vykonali na vytvorenie a implementáciu webovej aplikácie na prípravu na vodičské skúšky. Popisujeme jednotlivé fázy vývoja, nastavenia prostredia, implementáciu kľúčových funkcií a spôsob spracovania dát.

3.1 Vytvorenie projektu a základné nastavenia aplikácie

Naša aplikácia je navrhnutá ako full-stack riešenie, ktoré kombinuje frontendovú časť postavenú na Reacte s TypeScriptom a backendovú časť využívajúcu Express.js, Prisma a Supabase. Tento prístup nám umožňuje vytvoriť efektívnu a škálovateľnú aplikáciu s jasnou deľbou medzi klientskou a serverovou logikou. Začali sme vytvorením hlavného adresára projektu a inicializáciou Node.js prostredia: `mkdir my-app ,cd my-app, npm init -y`.

V rámci hlavného adresára sme vytvorili priečinok pre frontendovú časť aplikácie. Použili sme Create React App s TypeScript šablónou na inicializáciu projektu: `npx create-react-app client --template typescript`. Po korektnej inicializácii sme nainštalovali Tailwind CSS pre štylovanie komponentov: `cd client ,npm install -D tailwindcss, npx tailwindcss init`. V súbore `tailwind.config.js` ktorý sa nám vytvoril po inštalácii sme nastavili cesty k súborom, v ktorých budeme používať Tailwind.

```
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "CommonJS",
    "outDir": "./dist",
    "typeRoots": [
      "./node_modules/@types",
      "./src/types"
    ],
    "esModuleInterop": true,
    "strict": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules"]
}
```

Obrázok 5 Ukážka `tailwind.config.js`

Do súboru `src/index.css` vytvoreného pri inicializácii Reactu sme na začiatok pridali základné a potrebné direktívy pre Tailwind: `@tailwind base; @tailwind components; @tailwind utilities;`

V hlavnom adresári projektu sme vytvorili priečinok pre backendovú časť. Inicializovali sme ho ako samostatný Node.js projekt a nainštalovali potrebné závislosti: `mkdir server`, `cd server`, `npm init -y`, `npm install express prisma @prisma/client`. Prismu sme si vybrali ako nástroj pre prácu s našimi tabuľkami so sprostredkovateľom postgresql. Inicializovali sme ju pomocou príkazu: `npx prisma init`. Týmto krokom sa vytvoril priečinok `prisma` so súborom `schema.prisma`, kde sme definovali databázové modely. Pre pripojenie k Supabase databáze sme v súbore `.env` nastavili premennú `DATABASE_URL` s hodnotou poskytnutou Supabase platformou keďže Supabase a Prisma už majú predvyrobenú integráciu.

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          Int           @id @default(autoincrement())
  email       String        @unique
  role        String        @default("USER")
  createdAt   DateTime       @default(now())
  updatedAt   DateTime       @updatedAt
  username    String?       @unique
  badges      Badge[]
  tests       Test[]
  questionStats QuestionStat[]
  userAnswers UserAnswer[]
  reports     Reports[]
}
```

Obrázok 6 Ukážka `schema.prisma`

Po definovaní modelov sme vykonali migráciu databázy pomocou príkazu `npx prisma migrate dev --name init`. Po migrácii sa nám vytvorili tabuľky podľa modelov v `schema.prisma` v našom projekte ktorý sme si vytvorili vo webovom rozhraní supabase.

3.2 Integrácia frontendu a backendu

Po nastavení a úspešnej inštalácii frontendu a backendu sme pristúpili k ich integrácii, aby sme zabezpečili plynulú komunikáciu medzi klientskou a serverovou časťou aplikácie. Na strane frontendu sme implementovali funkcie, ktoré odosielajú HTTP požiadavky (GET, POST, PATCH) na definované API endpointy nášho Express servera. Na strane backendu sme v rámci Express aplikácie definovali routy, ktoré smerujú požiadavky na príslušné funkcie vytvorené v systematicky rozdelených a pomenovaných controlleroch. Každá funkcia v controlleri spracováva prichádzajúce požiadavky, vykonáva potrebné operácie, ako napríklad interakciu s databázou prostredníctvom Prisma ORM, a následne vracia odpoveď frontendu. Tento prístup nám umožnil udržať kód organizovaný a modulárny, pričom sme zabezpečili jasné oddelenie zodpovedností medzi jednotlivými komponentmi aplikácie.

```
const router: Router = Router();

router.post('/user', handleNewUser);
router.get('/user', getUserByEmail);
router.get('/question/:id', getQuestionById);

router.get('/admin/users', authenticate, isAdmin, getAllUsers);
router.put('/admin/users/:id', authenticate, isAdmin, updateUser);
router.delete('/admin/users/:id', authenticate, isAdmin, deleteUser);

router.put('/users/update-username', authenticate, updateUsername);
```

Obrázok 7 Ukážka express routov

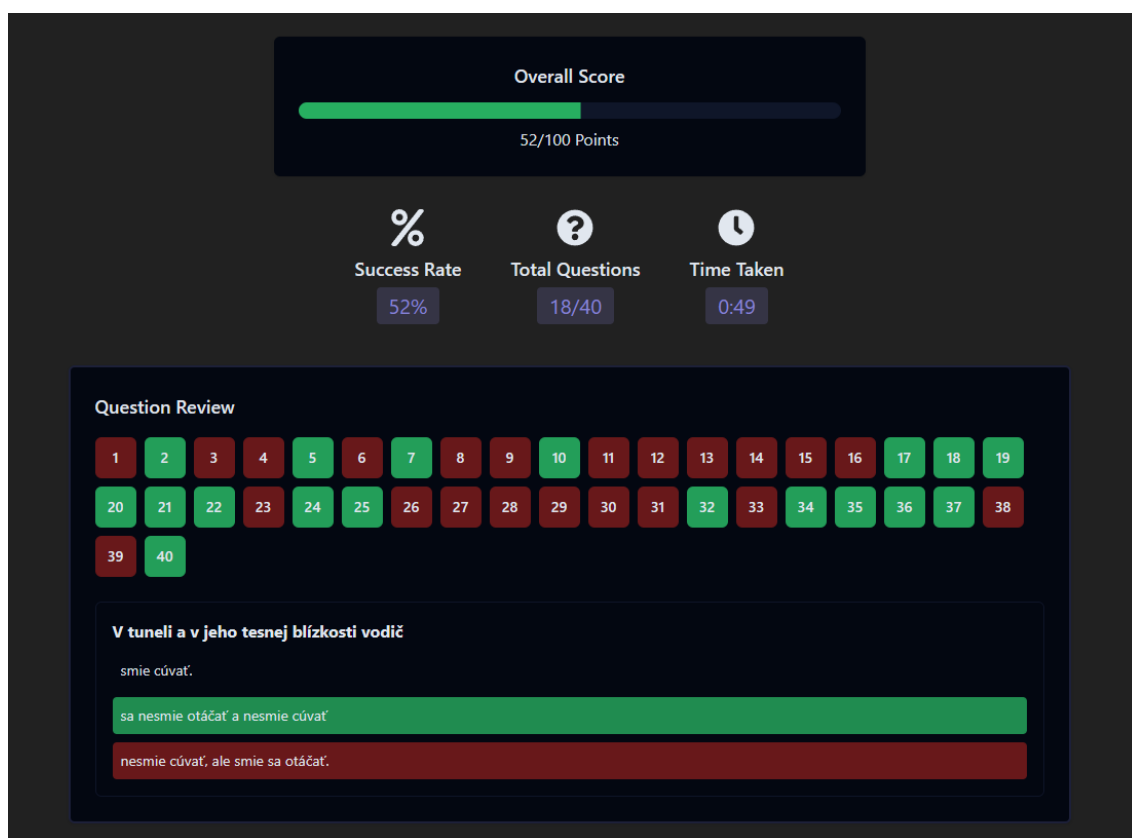
3.3 Vytváranie prvkov a funkcií aplikácie

V tejto kapitole sa zameriame na implementáciu kľúčových funkcií našej aplikácie, ktoré sú navrhnuté tak, aby poskytovali používateľom dynamické a prispôsobené skúšobné skúsenosti.

Po spustení testu sa pre používateľa dynamicky vygeneruje test pozostávajúci z otázok vybraných z databázy ktoré sme si pridali do tabuľky podľa oficiálnych otázok publikovaných ministerstvom vnútra Slovenskej republiky. Výber otázok je založený pre každý individuálny test na základe kombinácii náhodného výberu a analýzy predchádzajúcich výsledkov používateľa. Ak má používateľ históriu testov, systém identifikuje oblasti, v ktorých mal najväčšie ťažkosti, a uprednostňuje otázky z týchto

kategórií, aby sa zameral na zlepšenie slabších stránok. Tento prístup zabezpečuje personalizované a efektívne učenie. Aby sme ale testy ponechali naturálne, a nedávali používateľovi stále jeho zlé otázky, tak pridáme šancu či vôbec budú tieto otázky vybraté. Každá kategória má individuálny maximálny počet zlých otázok.

Počas testu sú všetky odpovede používateľa zaznamenávané. Užívateľ má k dispozícii jednotlivé otázky nahlásiť a popísať chybu alebo dôvod nahlásenia. Po odovzdaní testu systém vyhodnotí odpovede a identifikuje nesprávne zodpovedané otázky. Informácie o týchto nesprávnych odpovediach sa ukladajú do databázovej tabuľky, ktorá sleduje výkonnosť používateľa. Táto tabuľka obsahuje údaje ako id používateľa, id otázky, dátum testu a ďalšie relevantné metriky. Tieto údaje sú následne analyzované na generovanie štatistík o výkonnosti používateľa, identifikáciu oblastí na zlepšenie a prispôsobenie budúcich testov tak, aby sa zamerali na oblasti, kde má používateľ najväčšie rezervy. Tento proces umožňuje neustále prispôsobovanie obsahu testov individuálnym potrebám používateľa, čo vedie k efektívnejšiemu učeniu a lepšej príprave na vodičské skúšky.



Obrázok 8 Vyhodnotenie testu s možnosťou prezerania otázok

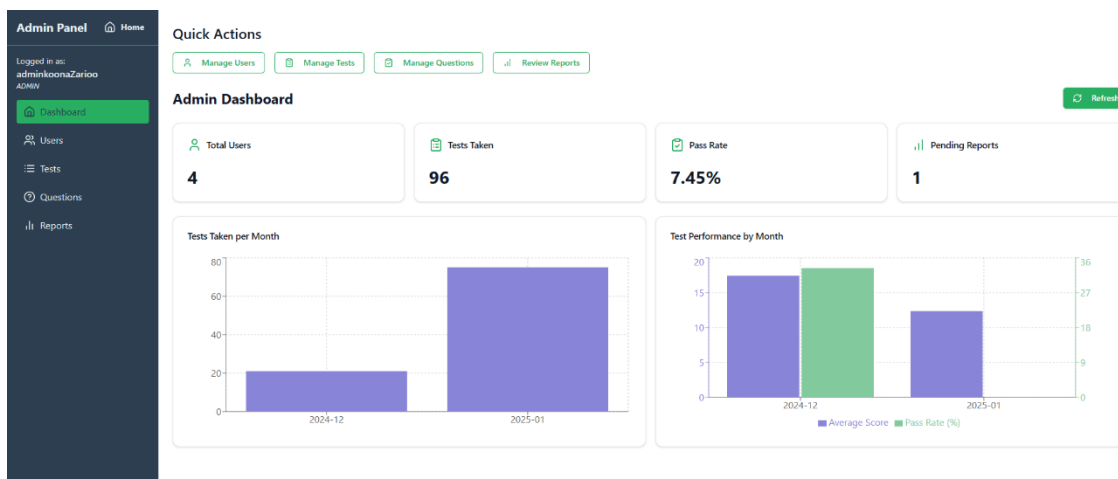
Administrátorský panel našej webovej aplikácie je prístupný výhradne autorizovaným používateľom s pridelenou rolou administrátora. Tento panel poskytuje komplexné nástroje na efektívnu správu obsahu a monitorovanie aktivity používateľov.

Administrátor má možnosť rozširovať databázu otázok pridávaním nových položiek, pričom pri vytváraní novej otázky môže zadať jej text, definovať viacero možných odpovedí a označiť správnu odpoveď. Taktiež môže priradiť otázku k určitej kategórii alebo téme, čo uľahčuje jej následné vyhľadávanie a organizáciu. V prípade potreby môže administrátor upravovať už existujúce otázky, meniť ich text, aktualizovať odpovede alebo meniť správnu odpoveď, ako aj aktualizovať priradenie otázky k inej kategórii. Ak je niektorá otázka zastaraná, nepresná alebo inak nevhodná, administrátor ju môže odstrániť z databázy.

Administrátor má prístup k detailným štatistikám o výkonnosti jednotlivých používateľov, môže si prezerať históriu absolvovaných testov každého používateľa, vrátane dátumov, dosiahnutých skóre a času stráveného na jednotlivých testoch. Tieto informácie sú prezentované v prehľadných grafoch a tabuľkách pre ľahšiu analýzu. Na základe zhromaždených dát môže administrátor identifikovať oblasti, v ktorých majú používatelia najväčšie problémy, napríklad zistiť, ktoré otázky sú najčastejšie zodpovedané nesprávne, alebo ktoré témy robia používateľom najväčšie ťažkosti. Tieto informácie môžu byť použité na zlepšenie obsahu alebo na vytvorenie dodatočných vzdelávacích materiálov.

Používatelia majú možnosť poskytovať spätnú väzbu týkajúcu sa otázok. Administrátor má prístup k týmto pripomienkam prostredníctvom administrátorského panela, kde sú zoradené podľa dátumu prijati. Administrátor môže priamo reagovať na poskytnutú spätnú väzbu, čo môže zahŕňať riešenie nahlásených problémov alebo implementáciu navrhovaných vylepšení. Týmto spôsobom sa zabezpečuje neustále zlepšovanie otázok a zvyšovanie spokojnosti používateľov.

Tieto funkcie administrátorského panela umožňujú komplexnú správu obsahu, monitorovanie výkonnosti používateľov a aktívnu komunikáciu s nimi, čo prispieva k neustálemu zlepšovaniu kvality poskytovaných služieb.



Obrázok 9 Ukážka admin dashboardu

3.4 Hosting a Deployment

Pri hostovaní a nasadení nášho projektu sme využili moderné a bezplatné služby, ktoré nám umožňujú efektívne spravovať a škálovať našu aplikáciu. Našou prioritou bolo vytvoriť infraštruktúru, ktorá je jednoduchá na údržbu a zároveň nákladovo efektívna. Kombináciou moderných nástrojov a služieb sme dosiahli stabilnú infraštruktúru, ktorá je pripravená na budúce rozšírenia. V tejto kapitole podrobne popíšeme, ako sme postupovali pri nasadzovaní aplikácie.

Doménu pre náš projekt sme zakúpili cez Namecheap, čo nám poskytlo spoľahlivú platformu na správu DNS záznamov. Aby sme zabezpečili správne prepojenie našej domény s frontendom, backendom a emailovými službami, nastavili sme nasledujúce DNS záznamy podľa priloženého obrázka.

☐	Type	Host	Value	TTL	
☐	A Record	@	76.76.21.21	Automatic	
☐	CNAME Record	50228900	sendgrid.net.	Automatic	
☐	CNAME Record	api	drive-edu.onrender.com.	Automatic	
☐	CNAME Record	em6996	u50228900.wl151.sendgrid.net.	Automatic	
☐	CNAME Record	s1_domainkey	s1.domainkey.u50228900.wl151.sendgrid.net.	Automatic	
☐	CNAME Record	s2_domainkey	s2.domainkey.u50228900.wl151.sendgrid.net.	Automatic	
☐	CNAME Record	url9748	sendgrid.net.	Automatic	
☐	CNAME Record	www	cname.vercel-dns.com.	Automatic	
☐	TXT Record	_dmarc	v=DMARC1: p=none;	Automatic	

Obrázok 10 DNS konfigurácia na platforme Namecheap

Frontend našej aplikácie hostujeme na platforme Vercel, ktorá nám ponúka bezplatné a jednoduché riešenie na nasadenie moderných webových aplikácií. Medzi hlavné výhody, ktoré sme využili, patria: automatické nasadzovanie pri každom pushnutí zmien do hlavnej vetvy repozitára na GitHub, HTTPS certifikáty generované automaticky, čo zabezpečuje šifrovanú komunikáciu a jednoduché prepojenie cez subdoménu `www.pomocou` CNAME záznamu. Vercel nám poskytuje rýchlu spätnú väzbu pri vývoji a zároveň spoľahlivé prostredie pre prevádzku aplikácie.

Pre backend sme zvolili platformu Render, ktorá nám umožňuje hostovať serverové aplikácie bezplatne a bez zbytočných komplikácií. Na Render sme nasadili našu aplikáciu s nasledujúcimi benefitmi: automatizované nasadzovanie zmien pri aktualizácii backendového repozitára a stabilné prepojenie medzi frontendom a backendom cez subdoménu `api` (DNS záznam), ktorá smeruje na `drive-edu.onrender.com`. Render nám umožňuje bezpečnú a stabilnú prevádzku backendu bez potreby manuálnej správy serverov.

Na zabezpečenie e-mailovej komunikácie sme využili službu SendGrid, ktorá nám poskytuje možnosť odosielať transakčné e-maily, ako napríklad resetovanie hesiel alebo autorizáciu. Pri konfigurácii sme postupovali nasledovne: nastavili sme DKIM autentifikáciu pomocou CNAME záznamov (`s1._domainkey`, `s2._domainkey`) a pridali sme TXT záznam pre validáciu DMARC, čím sme zvýšili dôveryhodnosť našich e-mailov. Použili sme vlastný e-mailový alias na odosielanie správ, čo zabezpečilo profesionálny vzhľad komunikácie. SendGrid nám umožňuje spoľahlivo odosielať e-maily s vysokou doručiteľnosťou a poskytuje detailné štatistiky o ich výkone.

4 VÝSLEDKY PRÁCE A DISKUSIA

Počas vývoja aplikácie sme narazili na viacero problémov, ktoré sme museli riešiť. Jedným z najväčších bola integrácia Supabase ako backendovej služby. Aj keď Supabase poskytuje rozhranie podobné Firebase, narazili sme na obmedzenia pri správe autentifikácie a prístupu k databázovým záznamom. Niektoré operácie, ako filtrovanie dát na základe používateľských oprávnení, si vyžadovali dodatočné úpravy v politike zabezpečenia a priamu komunikáciu s databázou cez Prisma ORM.

Ďalšou výzvou bolo nasadenie aplikácie. Frontend sme hostovali na Vercel, ktorý umožňuje jednoduché nasadenie z GitHub repozitára, avšak pri komunikácii s backendom na Render sme sa stretli s problémami CORS. Museli sme manuálne upraviť hlavičky HTTP odpovedí a správne nakonfigurovať middleware v Node.js a Express, aby API požiadavky fungovali správne. Okrem toho sa ukázalo, že Render v bezplatnom pláne je celkom pomalý a po určitom čase uspáva aplikáciu, čo spôsobovalo dlhšie odozvy pri prvom volaní API.

Celkovo nám vývoj tejto aplikácie priniesol mnoho cenných skúseností. Museli sme sa prispôbiť technickým obmedzeniam hostingu, riešiť bezpečnostné otázky pri práci s používateľskými údajmi a optimalizovať výkon aplikácie. Napriek problémom sa nám podarilo vytvoriť funkčnú webovú aplikáciu, ktorá spĺňa stanovené požiadavky.

5 ZÁVERY PRÁCE

Hlavným cieľom tejto práce bolo vytvoriť komplexnú webovú aplikáciu na prípravu používateľov na vodičské skúšky na Slovensku. Podarilo sa nám vyvinúť funkčný systém, ktorý poskytuje simulované testy podľa reálnych skúšobných podmienok, umožňuje precvičovanie dopravných značiek prostredníctvom flashcards a obsahuje interaktívny simulátor križovatiek. Týmto spôsobom aplikácia ponúka efektívny nástroj na prípravu budúcich vodičov.

Dôležitým prvkom bolo navrhnutie inteligentného testovacieho systému, ktorý analyzuje chyby používateľov a prispôsobuje testy ich individuálnym potrebám. Tento mechanizmus umožňuje efektívnejšie učenie a zvyšuje pravdepodobnosť úspešného zvládnutia skúšky. Používatelia majú k dispozícii prehľadné štatistiky o svojom výkone, čo im pomáha identifikovať slabé miesta a systematicky sa zlepšovať.

Pri vývoji sme kladli dôraz na prehľadné a intuitívne užívateľské rozhranie, ktoré umožňuje jednoduché ovládanie a efektívne učenie. Použitím moderných technológií, ako React, Tailwind CSS a ShadCN, sme vytvorili responzívny dizajn, ktorý je prispôsobený aj pre mobilné zariadenia.

Okrem hlavnej časti aplikácie sme implementovali administrátorský panel, ktorý umožňuje správu testov, sledovanie štatistík a spracovanie spätnej väzby od používateľov. Týmto spôsobom môžeme aplikáciu neustále vylepšovať a prispôbovať potrebám používateľov.

Celkovo sa nám podarilo dosiahnuť stanovené ciele, pričom aplikácia ponúka ucelený systém na prípravu na skúšky. Do budúcnosti by bolo možné rozšíriť funkcionality o pokročilejšie analytické nástroje na sledovanie progresu alebo integráciu s ďalšími vzdelávacími platformami. Napriek niektorým technickým výzvam sme vytvorili aplikáciu, ktorá môže významne pomôcť používateľom pri zvládnutí vodičských skúšok a zlepšiť tak bezpečnosť na cestách.

6 ZHRNUTIE

Cieľom tejto práce bolo vytvoriť webovú aplikáciu na efektívnu prípravu používateľov na vodičské skúšky na Slovensku. Aplikácia obsahuje simulované testy, interaktívny simulátor križovatiek a precvičovanie dopravných značiek prostredníctvom flashcards. Implementovali sme inteligentný testovací systém, ktorý analyzuje chyby používateľov a prispôsobuje testy ich výkonu, čím zefektívňuje učenie.

Na vývoj sme použili moderné technológie ako React, TypeScript, Node.js, Express a Supabase s databázou PostgreSQL. Užívateľské rozhranie sme navrhli s dôrazom na jednoduchosť a prehľadnosť. Administrátorský panel umožňuje správu otázok, sledovanie štatistík a spracovanie spätnej väzby.

Aplikácia úspešne naplňa svoj účel a poskytuje komplexný nástroj na prípravu na skúšky. Do budúcnosti by bolo možné rozšíriť funkcionality o pokročilú analýzu výkonu či integráciu s ďalšími vzdelávacími platformami.

ZOZNAM POUŽITEJ LITERATÚRY

[1] REACT. 2025 [online]. [cit. 31.1.2025].

Dostupné na internete: <https://react.dev/>

[2] TYPESCRIPT LANG. 2025. CSS. [online]. [cit. 31.1.2025].

Dostupné na internete: <https://www.typescriptlang.org/>

[3] DIGITAL OCEAN. 2025. How To Build a REST API with Prisma and PostgreSQL.

[online]. [cit. 31.1.2025]. Dostupné na internete:

<https://www.digitalocean.com/community/tutorials/how-to-build-a-rest-api-with-prisma-and-postgresql>

[4] MEDIUM. 2025. Integrating Prisma ORM with PostgreSQL and Express. [online].

[cit. 31.1.2025].

Dostupné na internete: <https://medium.com/@codethematrix/integrating-prisma-orm-with-postgresql-and-express-3f7b726cecec>

[5] FREDABOD. 2025. A Simple CRUD App With Prisma, Express, and PostgreSQL.

[online]. [cit. 31.1.2025]. Dostupné na internete: <https://dev.to/fredabod/a-simple-crud-app-with-prisma-express-and-postgresql-4on4>

[6] GIT. 2025. Free and open source distributed version control system. [online]. [cit.

31.1.2025].

Dostupné na internete: <https://git-scm.com/>

[7] MDN DOCS. [online]. 2025 [cit. 31.1.2025]. Dostupné na internete:

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs

[8] GITHUB. Build and ship software on a single, collaborative platform. [online]. 2025

[cit. 31.1.2025]. Dostupné na internete: <https://github.com/>

[9] GEEKSFORGEEKS. 2025. Introduction to Tailwind CSS.[online]. [cit. 31.1.2025].

Dostupné na internete: <https://www.geeksforgeeks.org/introduction-to-tailwind-css/>

[10] THREEJS. 2025. JavaScript 3D Library. [online]. [cit. 31.1.2025]. Dostupné na

internet: <https://threejs.org/>

[10] REACT TYPESCRIPT CHEATSHEET. 2025. [online]. [cit. 31.1.2025]. Dostupné

na internete: <https://react-typescript-cheatsheet.netlify.app/docs/basic/setup>

PRÍLOHA A

USB príloha

Priložené USB obsahuje:

- Dokumentáciu v elektronickej podobe
- Zdrojový kód aplikácie
- Prezentácia k projektu